In [1]:
```
import cv2
import copy
import random
import gc
import gym
import numpy as np
import torch
import ipywidgets as widgets
import torch.nn.functional as F
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

from gym import spaces
from tqdm import tqdm
from collections import deque
from IPython import display
from IPython.display import clear_output
from matplotlib import animation

cv2.ocl.setUseOpenCL(False)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

In [21]:
```
class Replay_Buffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def store(self, state, action, new_state, reward, done):
        state = np.expand_dims(state, 0)
        new_state = np.expand_dims(new_state, 0)

        self.buffer.append([state, action, new_state, reward, done])

    def replay(self, batch_size):
        state, action, new_state, reward, done = zip(
            *random.sample(self.buffer, batch_size)
        )

        return np.concatenate(state), action, np.concatenate(new_state), reward,

    def __len__(self):
        return len(self.buffer)
```

In [42]:
```
epsilon_S = 1.0
epsilon_E = 0.01
epsilon_decay = 30000

_epsilon = lambda frame: epsilon_E + (epsilon_S - epsilon_E)*np.exp(-frame/epsil
# plt.plot([_epsilon(frame) for frame in range(1000000)]);
```

In [23]:
```
class NoopResetEnv(gym.Wrapper):
    def __init__(self, env, noop_max=30):
        gym.Wrapper.__init__(self, env)
        self.noop_max = noop_max
        self.override_num_noops = None
        self.noop_action = 0
        assert env.unwrapped.get_action_meanings()[0] == 'NOOP'
```

```
    def reset(self, **kwargs):
        self.env.reset(**kwargs)
        if self.override_num_noops is not None:
            noops = self.override_num_noops
        else:
            noops = self.unwrapped.np_random.integers(1, self.noop_max + 1) #pyl
        assert noops > 0
        obs = None
        for _ in range(noops):
            obs, _, done, _, info = self.env.step(self.noop_action)
            if done:
                obs = self.env.reset(**kwargs)
        return obs, info

    def step(self, ac):
        return self.env.step(ac)


def make_atari(env_id, render_mode=None):
    env = gym.make(env_id, render_mode = render_mode)
    assert 'NoFrameskip' in env.spec.id
    env = NoopResetEnv(env, noop_max=30)
    return env

class ImageToPyTorch(gym.ObservationWrapper):

    def __init__(self, env):
        super(ImageToPyTorch, self).__init__(env)
        old_shape = self.observation_space.shape
        self.observation_space = gym.spaces.Box(low=0.0, high=1.0, shape=(old_sh

    def observation(self, observation):
        return np.swapaxes(observation, 2, 0)


def wrap_pytorch(env):
    return ImageToPyTorch(env)
```

In [24]:
```
env_id = "PongNoFrameskip-v4"
env    = make_atari(env_id, render_mode='rgb_array')
env    = wrap_pytorch(env)
```

In [25]:
```
def compute_td_loss(batch_size, device):
    state, action, reward, next_state, done = replay_buffer.replay(batch_size)
    state      = torch.tensor(state).to(device)
    next_state = torch.tensor(np.array(next_state), requires_grad=False).to(devi
    action     = torch.LongTensor(action).to(device)
    reward     = torch.FloatTensor(reward).to(device)
    done       = torch.FloatTensor(done).to(device)

    q_values      = model(state)
    next_q_values = model(next_state)

    q_value          = q_values.gather(1, action.unsqueeze(1)).squeeze(1)
    next_q_value     = next_q_values.max(1)[0]
    expected_q_value = reward + gamma * next_q_value * (1 - done)

    loss = (q_value - expected_q_value.data).pow(2).mean()
```

```
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()
```

In [26]:
```
class CnnDQN(nn.Module):
    def __init__(self, input_shape, num_actions):
        super(CnnDQN, self).__init__()

        self.input_shape = input_shape
        self.num_actions = num_actions

        self.features = nn.Sequential(
            nn.Conv2d(input_shape[0], 32, kernel_size=8, stride=4),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1),
            nn.ReLU()
        )

        self.fc = nn.Sequential(
            nn.Linear(self.feature_size(), 512),
            nn.ReLU(),
            nn.Linear(512, self.num_actions)
        )

    def forward(self, x):
        x = x.float()
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x

    def feature_size(self):
        return self.features(torch.autograd.Variable(torch.zeros(1, *self.input_

    def act(self, state, epsilon):
        with torch.no_grad():
            if random.random() > epsilon:
                state   = torch.FloatTensor(state).unsqueeze(0)
                state = state.to(device)
                q_value = self.forward(state)
                action  = q_value.max(1)[1].item()
            else:
                action = random.randrange(env.action_space.n)
        return action
```

In [27]:
```
import torch
print(torch.cuda.is_available())  # True면 정상
print(torch.cuda.get_device_name(0))  # GPU 이름 출력
```

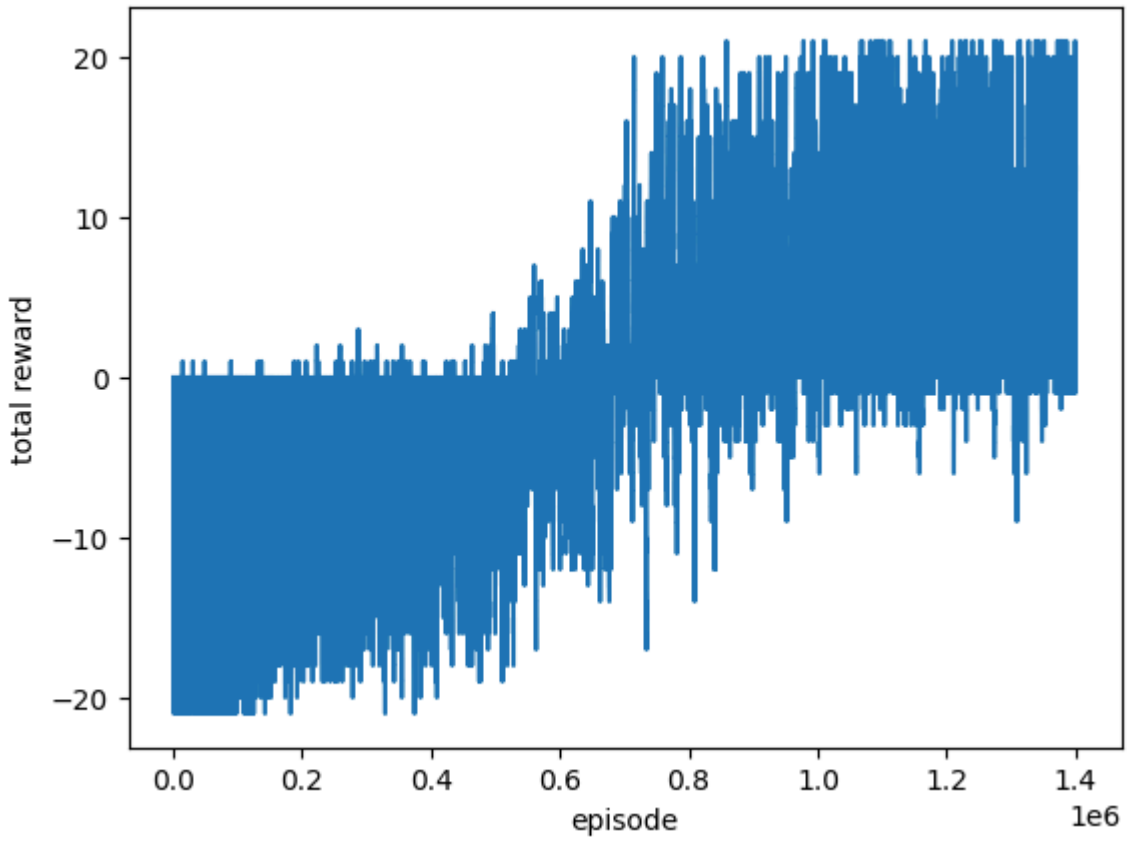
```
True
NVIDIA GeForce GTX 1650 Ti with Max-Q Design
```

In [28]:
```
model = CnnDQN(env.observation_space.shape, env.action_space.n)

model = model.cuda()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.00001)

replay_initial = 10000
replay_buffer = Replay_Buffer(100000)
```

In [30]:
```
num_frames = 1400000
batch_size = 32
gamma      = 0.99

losses = []
all_rewards = []
episode_reward = 0
step = 0
ep = 0
max_steps = 0

state, _ = env.reset()
for frame_idx in tqdm(range(1, num_frames + 1)):

    epsilon = _epsilon(frame_idx)
    action = model.act(state, epsilon)

    next_state, reward, done, _, _ = env.step(action)
    replay_buffer.store(state, action, reward, next_state, float(done))

    state = next_state
    episode_reward += reward
    step += 1

    if done:
        state, _ = env.reset()
        all_rewards.append(episode_reward)
        episode_reward = 0
        max_steps = max(step, max_steps)
        step = 0
        ep += 1
        gc.collect()

    if len(replay_buffer) > replay_initial:
        loss = compute_td_loss(batch_size, device)
        losses.append(loss)

    all_rewards.append(episode_reward)
    if frame_idx % 10000 == 0:
        print("episode : {}, total reward : {}".format(ep, episode_reward))
        rgb_array = env.render()

plt.xlabel('episode')
plt.ylabel('total reward')
plt.plot(range(len(all_rewards)), all_rewards)
plt.savefig('reward_plot.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
100%|██████████| 1400000/1400000 [4:46:48<00:00, 81.36it/s]
```



In [36]:
```
def save_frames_as_gif(frames, path='./', filename='gym_animation.gif'):
    plt.figure(figsize=(frames[0].shape[1] / 72.0, frames[0].shape[0] / 72.0), d

    patch = plt.imshow(frames[0])
    plt.axis('off')

    def animate(i):
        patch.set_data(frames[i])

    anim = animation.FuncAnimation(plt.gcf(), animate, frames = len(frames), int

    anim.save(path + filename, writer='imagemagick', fps=6)



state, _ = env.reset()
frames = []
for t in tqdm(range(1000)):
    frames.append(env.render())
    action = model.act(state, epsilon)
    state, _, done, _, _ = env.step(action)
    if done:
        break

save_frames_as_gif(frames)
clear_output(wait=True)
```

In [41]:
```
state, _ = env.reset()
done = False
total_reward = 0

while not done:
    # 탐험 없이 항상 최적 행동 선택
```

```
    action = model.act(state, epsilon=0.0)
    next_state, reward, done, _, _ = env.step(action)
    state = next_state
    total_reward += reward
    env.render()  # 화면 출력 (옵션)
print('Total reward: {}'.format(total_reward))
```

```
Total reward: 20.0
```

In [37]:
```
gif_file = './gym_animation.gif'
file = open(gif_file, "rb")
image = file.read()
widgets.Image(
    value=image,
    format='png',
    width=300,
    height=400,
)
```

Out[37]:
```
Image(value=b'GIF89a\xa0\x00\xd2\x00\x84\x00\x00\x90H\x11\xff\xff\xff\xec\xec\x
ec\xa5g:\xae~Z\xbd\x98|\\\xb9[\…
```

In [33]:
```
torch.save(model, 'pingpong_dqn_model.pth')
```

In [34]:
```
torch.save(model.state_dict(), 'pingpong_dqn.pth')
```